NoteMind Frontend API Integration Guide

This guide is based on the **current backend implementation** in Java controllers/services/DTOs and compares that behavior with existing docs where they differ.

---

1) Base URLs and `/api/v1` prefix

`application.properties` sets:

- `server.servlet.context-path=/api/v1`

So every controller path is prefixed with `/api/v1`.

Recommended base URLs

- Local web/dev tools: `http://localhost:8080/api/v1`
- Android emulator: `http://10.0.2.2:8080/api/v1`
- Physical device on same network: `http://<YOUR_COMPUTER_LAN_IP>:8080/api/v1`

If the context path is changed/removed in backend config, update frontend base URL accordingly.

---

2) Auth integration

2.1 Endpoints

Method	Path	Auth required
POST	`/auth/signup`	No
POST	`/auth/login`	No
POST	`/auth/refresh-token`	No
POST	`/auth/logout`	No (per security config `/auth/**` permitAll)

2.2 Request payloads

`POST /auth/signup` and `POST /auth/login`

```
{
  "username": "alice",
  "password": "strong-password"
}
```

`POST /auth/refresh-token` and `POST /auth/logout`

```
{
  "refreshToken": "plain-refresh-token"
}
```

2.3 Actual response envelope and auth DTOs

All successful responses are wrapped in `ApiResponse`:

```
{
  "success": true,
  "message": "...",
  "data": { },
  "timestamp": "2026-10-14T18:00:00Z"
}
```

Signup/Login `data` shape (`AuthResponseDto`):

```
{
  "username": "alice",
  "userId": 123,
  "jwtToken": "eyJ...",
  "refreshToken": "..."
}
```

Refresh `data` shape (`RefreshResponseDto`):

```
{
  "jwtToken": "eyJ...",
  "refreshToken": "rotated-refresh-token"
}
```

Logout success:

```
{
  "success": true,
  "message": "Logged out successfully",
  "data": null,
  "timestamp": "..."
}
```

2.4 Confirmed behavior vs older docs/examples

- Code returns `jwtToken`, not `token`.
- Code does **not** return `expiresIn`.
- Refresh token is rotated on refresh (new refresh token returned).
- `/auth/logout` is in `/auth/**`, so not blocked by security matcher.

2.5 Token storage and auth header

Use secure storage (Android EncryptedSharedPreferences/Keystore, web secure storage strategy). Always send:

`Authorization: ******

for non-auth endpoints.

2.6 401/refresh retry flow (frontend)

1. Send request with bearer access token.
2. If response is 401 (or auth-failure status in your environment), call `/auth/refresh-token` with stored refresh token.

3. Replace both stored `jwtToken` and `refreshToken` from refresh response.
4. Retry original request once.
5. If refresh fails (401), clear session and route to login.

---

3) Notes endpoints

3.1 Endpoints

Method	Path	Notes
GET	`/notes?page=0&size=10&sort=createdAt,desc`	Paginated list
GET	`/notes/{id}`	Get one
POST	`/notes`	Multipart create
PUT	`/notes/{id}`	Multipart update
DELETE	`/notes/{id}`	Delete
GET	`/notes/search?query=...`	Search
GET	`/notes/refresh-summary/{id}?type=SHORT`	Refresh summary

3.2 Note DTO shape (`NoteDto`)

```
{
  "id": 1,
  "title": "...",
  "content": "...",
  "summary": "...",
  "summaryType": "SHORT",
  "imageUrl": "https://...",
  "createdAt": "2026-10-14T12:00:00",
  "updatedAt": "2026-10-14T12:30:00",
  "tags": [
    { "id": 10, "name": "java" }
  ],
  "folderId": 5
}
```

3.3 Multipart create/update format

Content type: `multipart/form-data`

Required part names:
• `note` (stringified JSON for `NoteDto` fields)
• `image` (optional file)

`note` JSON should include at minimum:
• `title` (required by service)
• `content` (required by service)
• `summaryType` (optional; defaults to `SHORT`)
• optionally `imageUrl`, `folderId` depending on flow

Summary enum values (`SummaryType`):
• `SHORT`
• `DETAILED`
• `BULLET_POINTS`

3.4 List pagination/sorting

`GET /notes` supports Spring `Pageable` params:
• `page` default `0`
• `size` default `10`
• `sort` default by `createdAt` (you can pass `createdAt,desc` etc)

Returns `ApiResponse<Page<NoteDto>>` where `data` contains `content`, `pageable`, `totalElements`, `totalPages`, etc.

3.5 Search

`GET /notes/search?query=term` returns `ApiResponse<List<NoteDto>>`.

Confirmed difference from older examples: no `results` wrapper and no guaranteed `relevanceScore` field in DTO.

3.6 Summary refresh

Actual path in controller is:
• `GET /notes/refresh-summary/{id}?type=SHORT`

Confirmed difference from some docs/examples that show `/notes/{id}/refresh-summary`.

---

4) Folder endpoints

Method	Path	Description
POST	`/folder`	Create folder
GET	`/folder`	List folders without nested notes
GET	`/folder?includeNotes=true`	List folders with nested notes
GET	`/folder/{id}`	Get folder by id
PATCH	`/folder/{id}`	Rename folder
DELETE	`/folder/{id}`	Delete folder

Request DTO (`FolderRequestDto`)

```
{ "name": "Study" }
```

Response DTOs

Without notes (`FolderDtoWithoutNotes`):

```
{ "id": 1, "name": "Study" }
```

With notes (`FolderDto`):

```
{
```

```
  "id": 1,
  "name": "Study",
  "notes": [ { "id": 11, "title": "..." } ]
}
```

Implementation detail: when `includeNotes=true`, backend also appends a synthetic folder `{ "id": null, "name": "NotInFolder", ... }` for notes without a folder.

---

5) Flashcard endpoints

5.1 Endpoints

Method	Path	Description
POST	`/notes/{noteId}/flashcards/generate?count=5`	AI generate
POST	`/notes/{noteId}/flashcards/regenerate?count=5`	AI regenerate
GET	`/notes/{noteId}/flashcards`	List by note
GET	`/flashcards/{flashCardId}`	Get one
POST	`/notes/{noteId}/flashcards`	Manual create
PATCH	`/flashcards/{flashCardId}`	Update one
DELETE	`/flashcards/{flashCardId}`	Delete one
DELETE	`/notes/{noteId}/flashcards`	Delete all for note

5.2 Count limits

`count` query param for generate/regenerate:

- min `1`
- max `50`
- default `5`

5.3 DTOs

Manual create/update request (`FlashCardRequestDto`):

```
{
  "question": "What is X?",
  "answer": "X is ..."
}
```

Response item (`FlashCardDto`):

```
{
  "id": 1,
  "question": "What is X?",
  "answer": "X is ...",
  "noteId": 7,
  "createdAt": "2026-10-14T18:00:00"
}
```

Generate/regenerate/list responses are `ApiResponse<List<FlashCardDto>>`.

---

6) Quiz generation endpoint

Method	Path	Description
POST	`/notes/{noteId}/quiz/generate?count=5`	Generate MCQ quiz from note

Count limits:
- min `1`
- max `20`
- default `5`

Response is `ApiResponse<List<QuizQuestionDto>>`, where each question:

```
{
  "question": "...",
  "options": ["A", "B", "C", "D"],
  "correctAnswer": "B",
  "explanation": "..."
}
```

Frontend quiz flow (recommended)

1. User chooses count (1..20).
2. Call generate endpoint.
3. Render one question at a time from `data[]`.
4. Track selected answers locally.
5. Show score and explanation per question at result step.
6. Regenerate by calling endpoint again.

---

7) Common API behavior and integration guidance

7.1 ApiResponse envelope

Success shape:

```
{
  "success": true,
  "message": "Success or action message",
  "data": {},
  "timestamp": "ISO-8601 instant"
}
```

Error shape (from `ApiResponse.error` via global exception handler):

```
{
  "success": false,
  "message": "Error message",
  "data": null,
```

```
  "timestamp": "ISO-8601 instant"
}
```

7.2 Auth header

For protected routes:

`Authorization: ******

7.3 Common statuses (confirmed)

- `200 OK`: most success operations (including notes/folder create/update/delete and auth endpoints)
- `201 Created`: flashcard generate, flashcard manual create, quiz generate
- `400 Bad Request`: invalid note/folder/search term etc
- `401 Unauthorized`: invalid credentials/refresh token related exceptions
- `404 Not Found`: missing user/note/folder/flashcard
- `409 Conflict`: username/folder conflicts
- `502 Bad Gateway`: AI service exception
- `500 Internal Server Error`: unexpected exceptions

7.4 Validation/error handling guidance

- Always parse envelope first (`success`, `message`, `data`).
- For non-2xx, surface `message` to user-friendly UI messages.
- For `401` after protected calls, attempt token refresh once then retry.
- For `400` on note create/update, ensure `title` and `content` are non-empty.
- For folder create/rename, ensure `name` is non-empty.

7.5 JavaScript `fetch` examples

Authorized JSON request

```
async function apiGet(path, token) {
  const res = await fetch(`${BASE_URL}${path}`, {
    headers: { Authorization: `****** }
  });
  const body = await res.json();
  if (!res.ok || body.success === false) throw new Error(body.message ||
'Request failed');
  return body.data;
}
```

Multipart note create

```
async function createNote(note, imageFile, token) {
  const fd = new FormData();
  fd.append('note', JSON.stringify(note));
  if (imageFile) fd.append('image', imageFile);

  const res = await fetch(`${BASE_URL}/notes`, {
    method: 'POST',
```

```
      headers: { Authorization: `****** },
      body: fd
    });

    const body = await res.json();
    if (!res.ok || !body.success) throw new Error(body.message || 'Create
  failed');
    return body.data;
  }
```

Refresh + retry sketch

```
  async function withRefresh(requestFn) {
    try {
      return await requestFn();
    } catch (e) {
      if (!isAuthError(e)) throw e;
      await refreshTokens();
      return requestFn();
    }
  }
```

---

8) Endpoint reference table (concise)

Area	Method	Path
Auth	POST	`/auth/signup`
Auth	POST	`/auth/login`
Auth	POST	`/auth/refresh-token`
Auth	POST	`/auth/logout`
Notes	GET	`/notes`
Notes	GET	`/notes/{id}`
Notes	POST	`/notes`
Notes	PUT	`/notes/{id}`
Notes	DELETE	`/notes/{id}`
Notes	GET	`/notes/search?query=...`
Notes	GET	`/notes/refresh-summary/{id}?type=...`
Folders	POST	`/folder`
Folders	GET	`/folder`
Folders	GET	`/folder/{id}`
Folders	PATCH	`/folder/{id}`
Folders	DELETE	`/folder/{id}`
Flashcards	POST	`/notes/{noteId}/flashcards/generate?count=...`
Flashcards	POST	`/notes/{noteId}/flashcards/regenerate?count=...`
Flashcards	GET	`/notes/{noteId}/flashcards`
Flashcards	GET	`/flashcards/{flashCardId}`

Flashcards	POST	`/notes/{noteId}/flashcards`
Flashcards	PATCH	`/flashcards/{flashCardId}`
Flashcards	DELETE	`/flashcards/{flashCardId}`
Flashcards	DELETE	`/notes/{noteId}/flashcards`
Quiz	POST	`/notes/{noteId}/quiz/generate?count=...`

---

9) Frontend implementation checklist

- [ ] Configure `BASE_URL` for web (`localhost`), emulator (`10.0.2.2`), and device (LAN IP)
- [ ] Implement auth store with `jwtToken` + `refreshToken`
- [ ] Add auth interceptor to send `Authorization: ******`
- [ ] Implement refresh-token retry once on auth failure
- [ ] Implement notes list pagination/sort controls
- [ ] Implement notes multipart create/update using `note` + optional `image`
- [ ] Support `SummaryType` values: `SHORT`, `DETAILED`, `BULLET_POINTS`
- [ ] Implement summary refresh via `/notes/refresh-summary/{id}`
- [ ] Implement folder list/create/rename/delete and `includeNotes=true` mode
- [ ] Handle `NotInFolder` synthetic folder in includeNotes response
- [ ] Implement flashcard generate/regenerate/create/update/delete/delete-all
- [ ] Enforce flashcard count bounds 1..50 in UI
- [ ] Implement quiz generation flow and enforce quiz count bounds 1..20
- [ ] Parse `ApiResponse` envelope consistently for success/error UX

---

10) Source-of-truth notes

This guide prioritizes confirmed behavior from:

- Controllers under `src/main/java/com/dhaliwal/notemind/controller`
- DTOs under `src/main/java/com/dhaliwal/notemind/dto` and `security/.../dto`
- `ApiResponse`, `GlobalExceptionHandler`, `SecurityConfig`, and related auth services

Where previous documentation examples differ (field names, status assumptions, or route shape), this guide marks the backend implementation as authoritative.